CSE 554 Machine Learning Systems

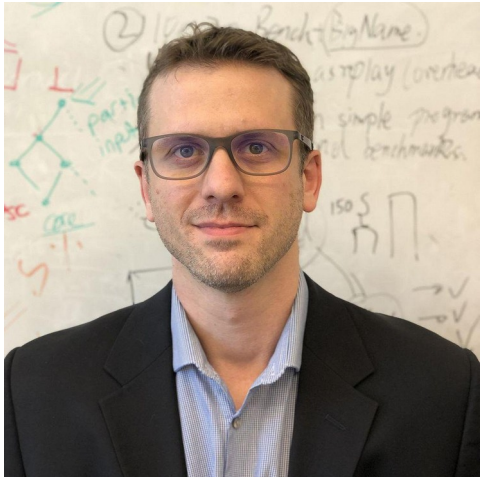
Lecture 1 – GPU Architecture and Introduction to GPU Programming

Baris Kasikci

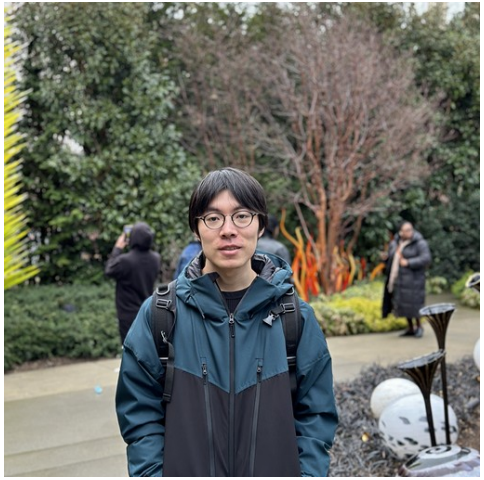
BE BOUNDLESS

Instructors

Baris Kasikci



Keisuke Kamahori

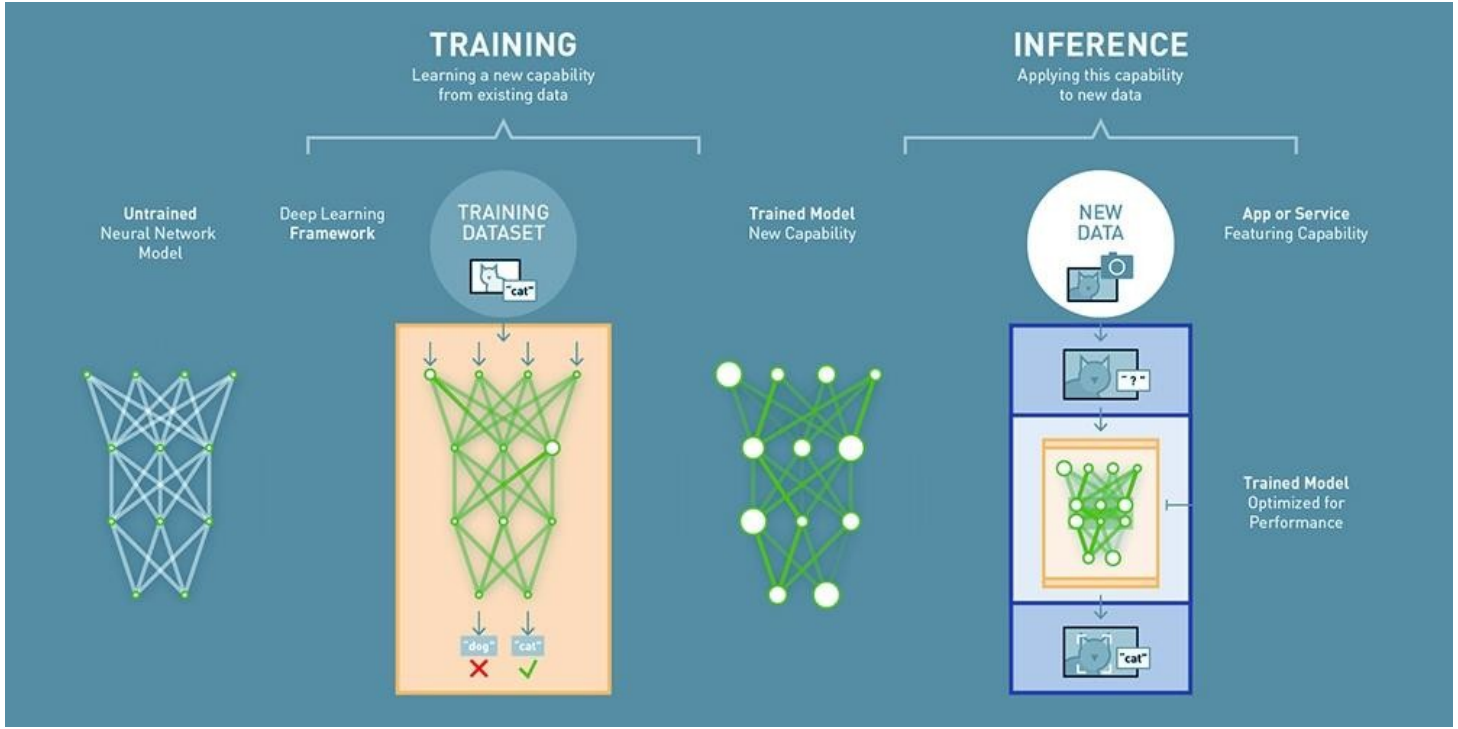


Shubham Tiwari

Prerequisites

- Basic C++ and Python programming
- Basic understanding of Deep Learning
  - Training/Inference workflow
    - Normalization (batch/layer)
    - Backpropagation
  - What qualifies?
    - A ML/DL course
      - CSE 446 or equivalent
    - Work/internship experience
    - Self-study
      - Zero to Hero by Andrej Karpathy

Training vs Inference



https://www.exxactcorp.com/blog/HPC/discover-the-difference-between-deep-learning-training-and-inference

Training learns from existing data
Inference applies learned capabilities to new data

Serving

Serving an ML model is about building a system to efficiently and scalably perform inference

- High throughput
- Low latency
- Compliant with diverse Service Level Objectives (SLOs)

LLMs Enable Many New Applications

AI Assistants (ChatGPT, Gemini)

Text-to-Image (DALL·E, MidJourney)

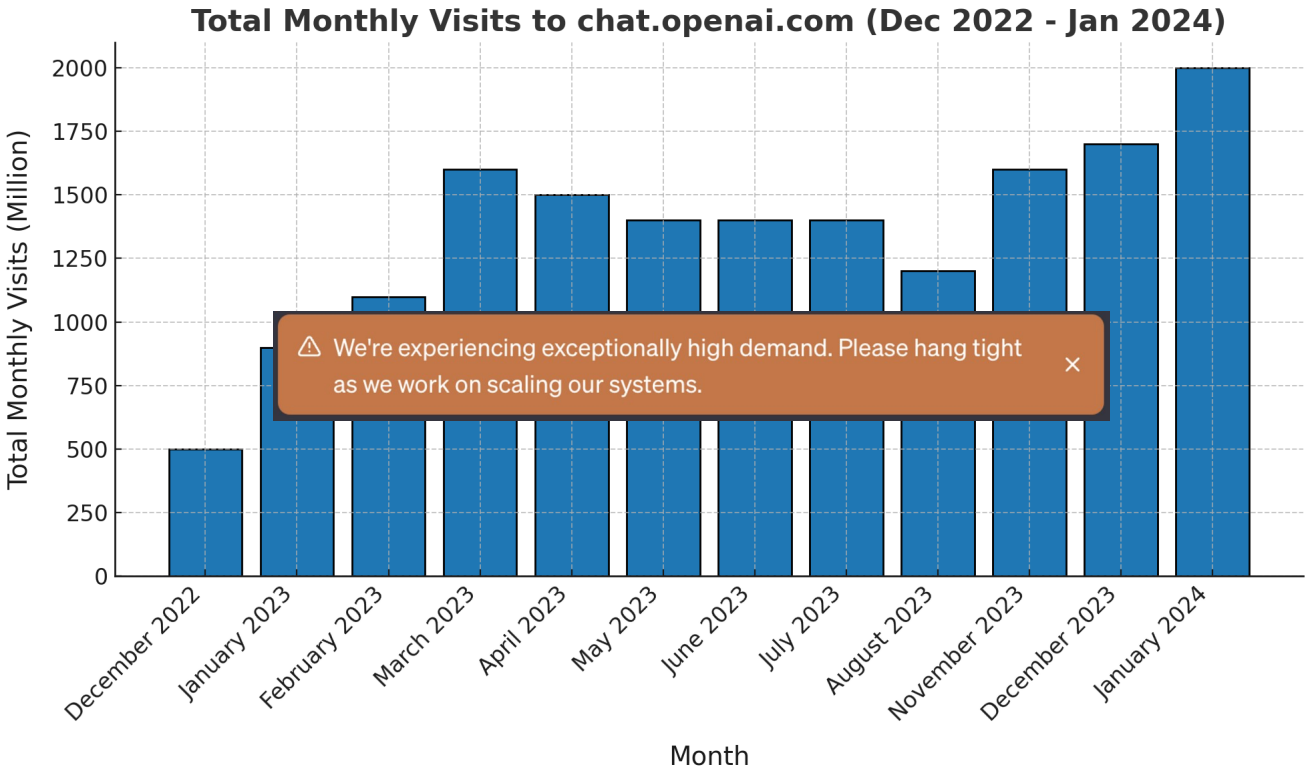
Creative Writing (Jasper, Copy.ai)

AI Coding Tools (GitHub Copilot, Replit)

Text-to-Speech & Audio (Descript, Synthesia)

Batching user requests and aiming for optimal throughput are key

LLM Serving Systems Face Surging Demand



https://nerdynav.com/chatgpt-statistics/

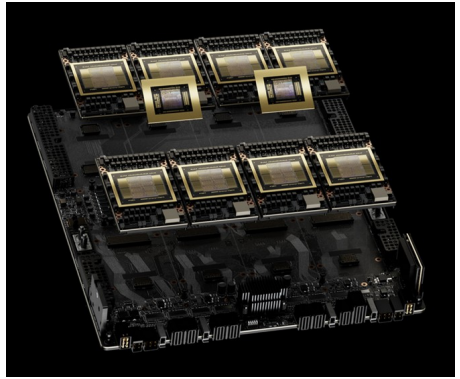
Cost of Training and Serving LLMs is Massive

Large-scale investment in H100s in 2024

- Meta: 300K[1]
- Google: 150K[1]
- Microsoft: 150K[1]
- X: 85K[2]

NVIDIA H200 HGX Server

- ~$250,000
- High operating cost
  - Up to ~10,000 W
- Long lead times



[1] Tom's Hardware
[2] The Register

Batching user requests and aiming for optimal throughput are key

Class Overview

1. GPU Architecture and Programming
2. Transformers (Architecture and Implementation)
3. Performance Modeling
4. Quantization and Sparsity
5. Memory Management
6. Mixture of Experts (MoE)
7. Retrieval-Augmented Generation (RAG)
8. Batching
9. Speculative Decoding
10. Parallelism
11. End-to-end Serving
12. Large-scale Deployment
13. Agentic Systems
14. RL for LLMs

Assignments and Grading

- 4 programming/profiling/analysis assignments
  - With short (2/3-page) reports
  - Done in groups of ~3
  - Each 23% of the grade
  - Done on CSE GPU servers
- Attendance and participation
  - 8% of the grade
    - 80%+ attendance & participation will get you the full 8%
- Please drop the class if you took 599k with me in Sp25

Group Formation

- We'll have groups of 3 in this class
- Use this doc to introduce yourself to others, communicate soon to start forming groups:
  - Or do it in person after class
- Once groups are formed, populate the appropriate tab in the doc

Setup Instructions

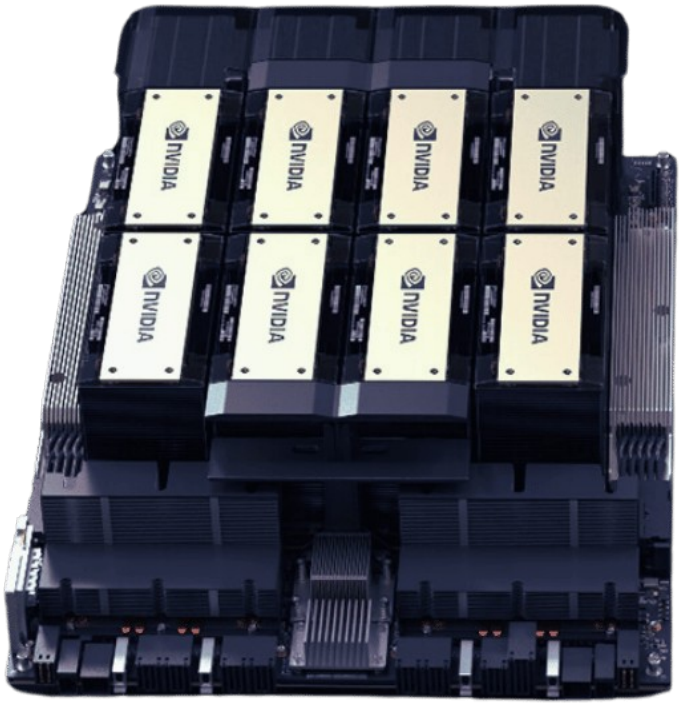
- Click on the following link for detailed GPU setup instructions: GPU Setup
  - ssh <cse_netid>@tempura.cs.washington.edu or
  - ssh <cse_netid>@tomago.cs.washington.edu
  - use CSE NetID password

Today

- Basics of the GPU Architecture
- Introduction to GPU programming
  - Pytorch
  - Triton
- Basics of performance analysis
  - Using profilers
  - Cross-checking profiling results analytically

What is a GPU?

- Graphics Processing Unit
- Originally Designed for accelerated graphics rendering
- Nowadays GPUs perform scientific computing and machine learning
- Come with software stacks
  - CUDA (NVIDIA), ROCm (AMD), etc.



https://www.carlexit.com/gigabyte-gpu-servers/

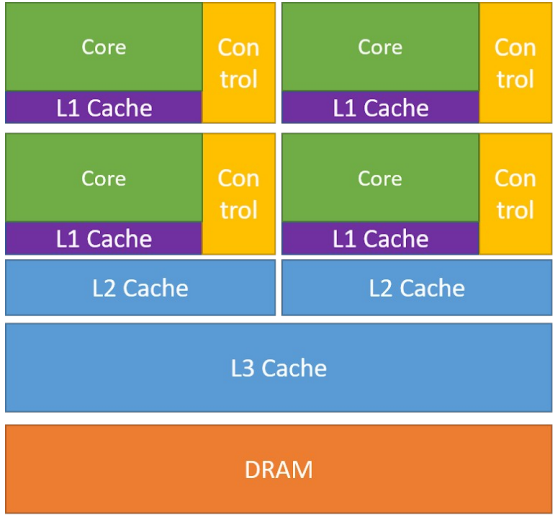
CPU vs GPU

CPU

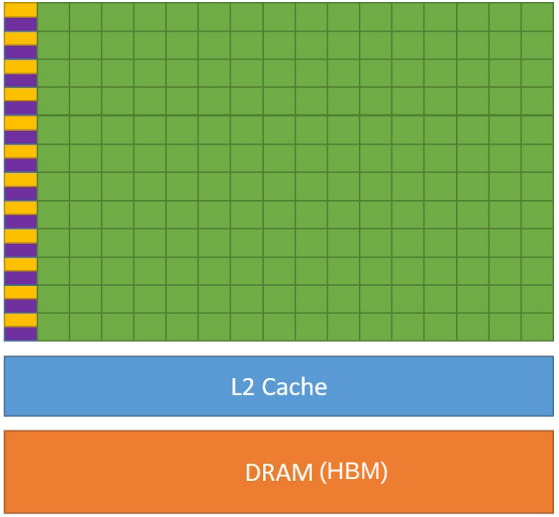
- Control logic (good with branching)
- Single thread performance

GPU

- Computation / loading
- Parallel processing



CPU



GPU

https://docs.nvidia.com/cuda/cuda-c-programming-guide/

Example: CPU vs GPU Specs

Specification	AMD EPYC™ 9555 (CPU)	NVIDIA H200 (GPU)
Cores / Threads	64 Cores / 128 Threads	16,896 CUDA Cores
Frequency	4.4 GHz	1.980 GHz
TFLOPs	~10-20 TFLOPs	989 TFLOPs
Memory Size	Up to 6 TB	144GB
Memory Bandwidth	576 GB/s	4800 GB/s
Memory Latency	~70ns	~110ns

CPU **DRAM**:
- Low latency, random access

GPU **HBM**:
- Higher bandwidth, structured batch access

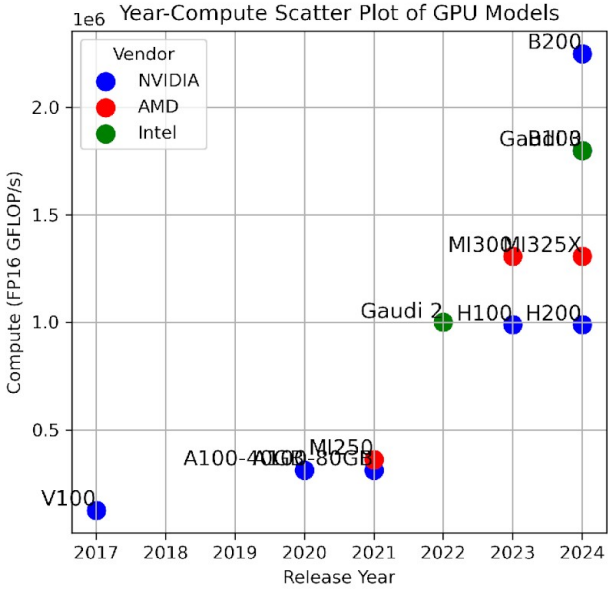
Example: CPU vs GPU Specs

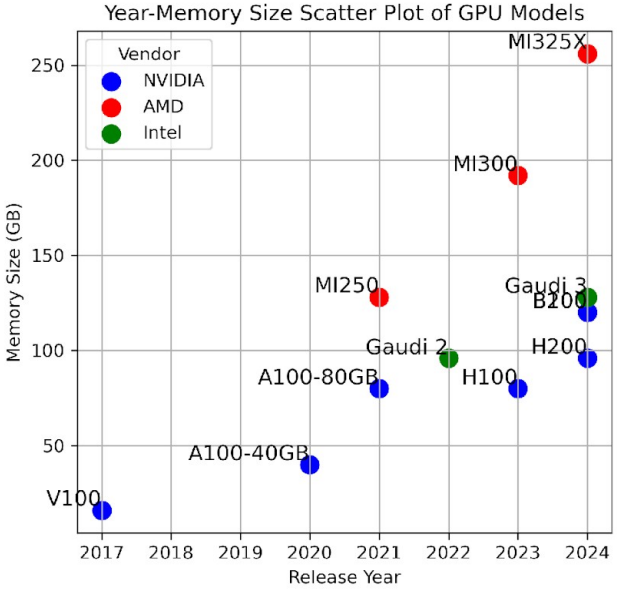
Specification	AMD EPYC™ 9555 (CPU)	NVIDIA H200 (GPU)
Cores / Threads	64 Cores / 128 Threads	16,896 CUDA Cores
Frequency	4.4 GHz	1.980 GHz
TFLOPs	~10-20 TFLOPs	989 TFLOPs
Memory Size	Up to 6 TB	144GB
Memory Bandwidth	576 GB/s	4800 GB/s
Memory Latency	~70ns	~110ns

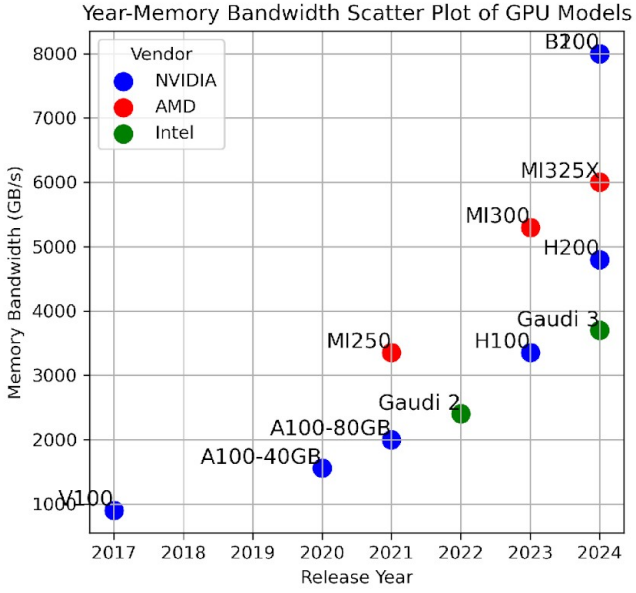
CPU **DRAM**:
- Low latency, random access

GPU **HBM**:
- Higher bandwidth, structured batch access

GPU vendors and models







https://imgur.com/a/O7N9klH

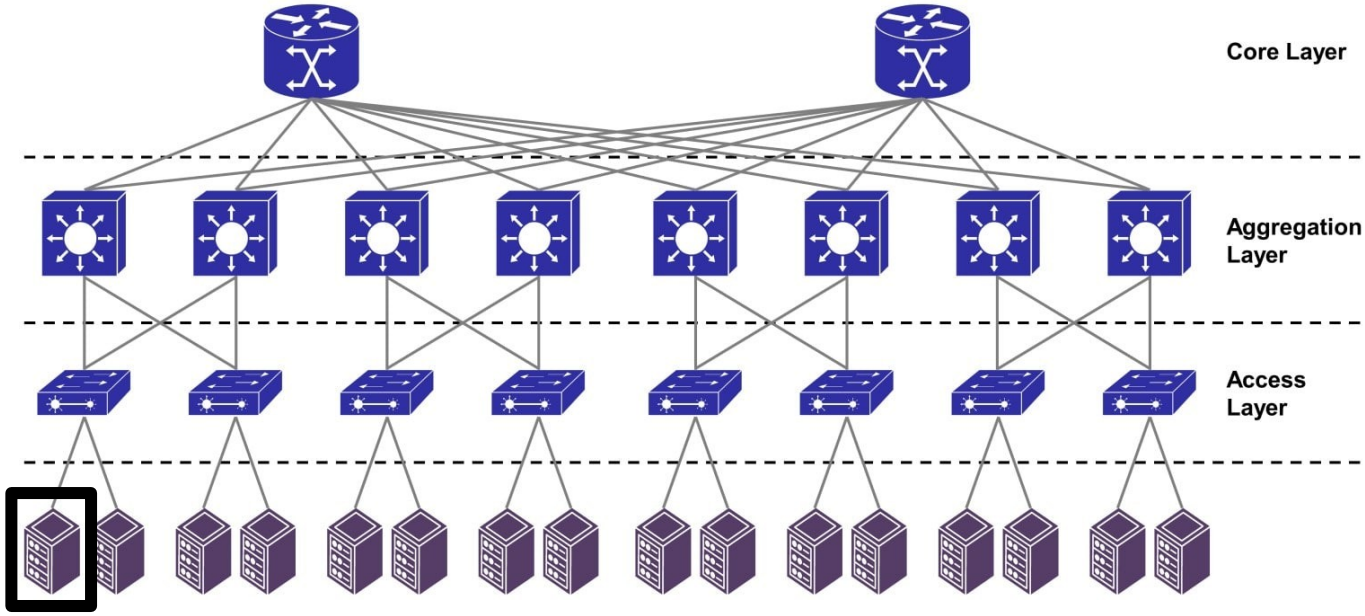
GPU white papers

It is very helpful to learn to navigate GPU whitepapers (contain tech specs, new features)

- A100
- H100
- 5090

- For this course, we will use an H100 in course material, but you will rely on Quadro RTX 6000 GPUs for assignments.

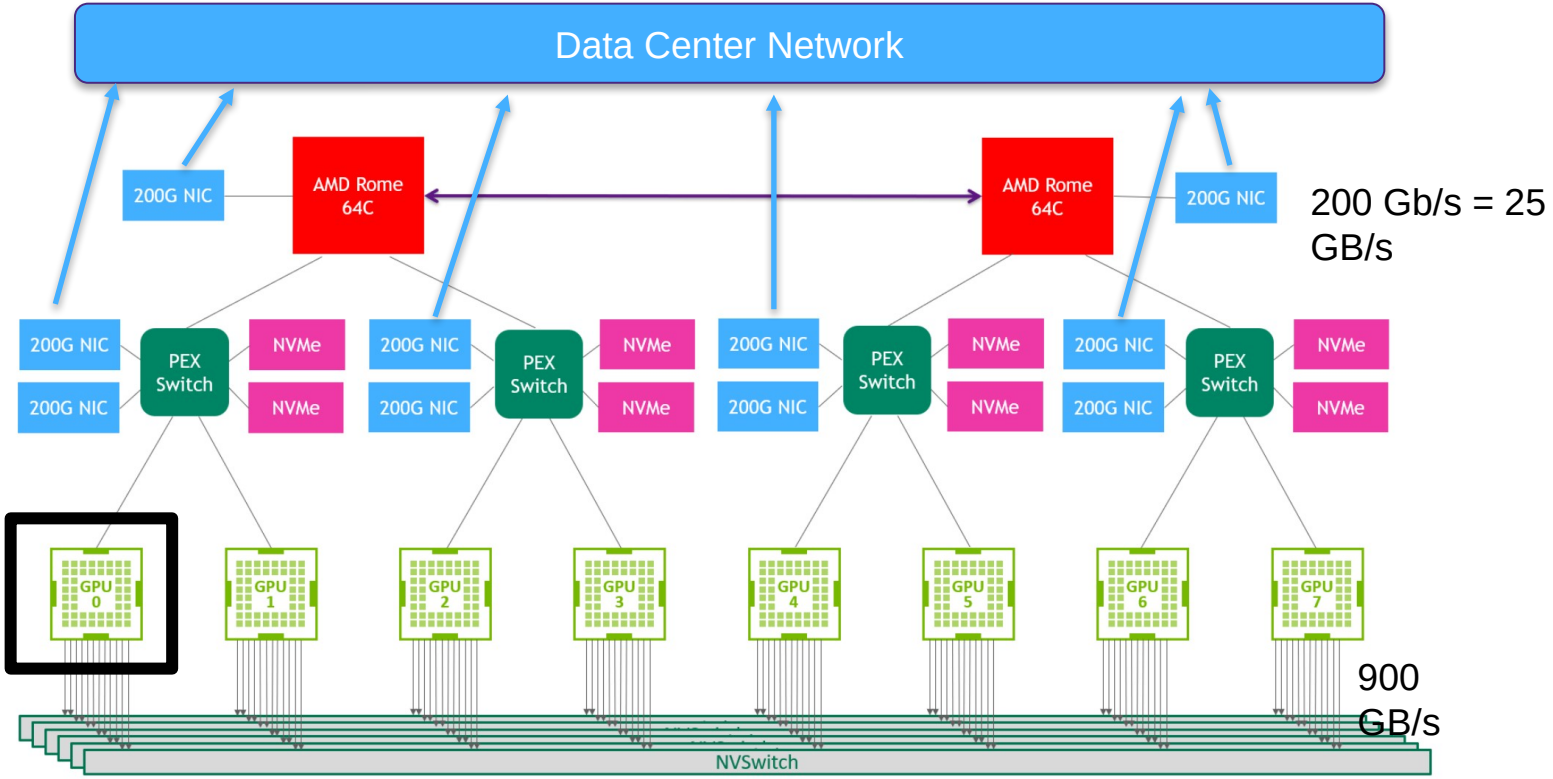
Data Center Network



GPU
Server

https://dgtlinfra.com/data-center-networking/

Non-Uniform Memory Access (NUMA) Effect



https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf

Latencies of accesses between GPU_0 – GPU_1 and GPU_0 – GPU_4 are not uniform

GPU Hardware Architecture (NVIDIA)

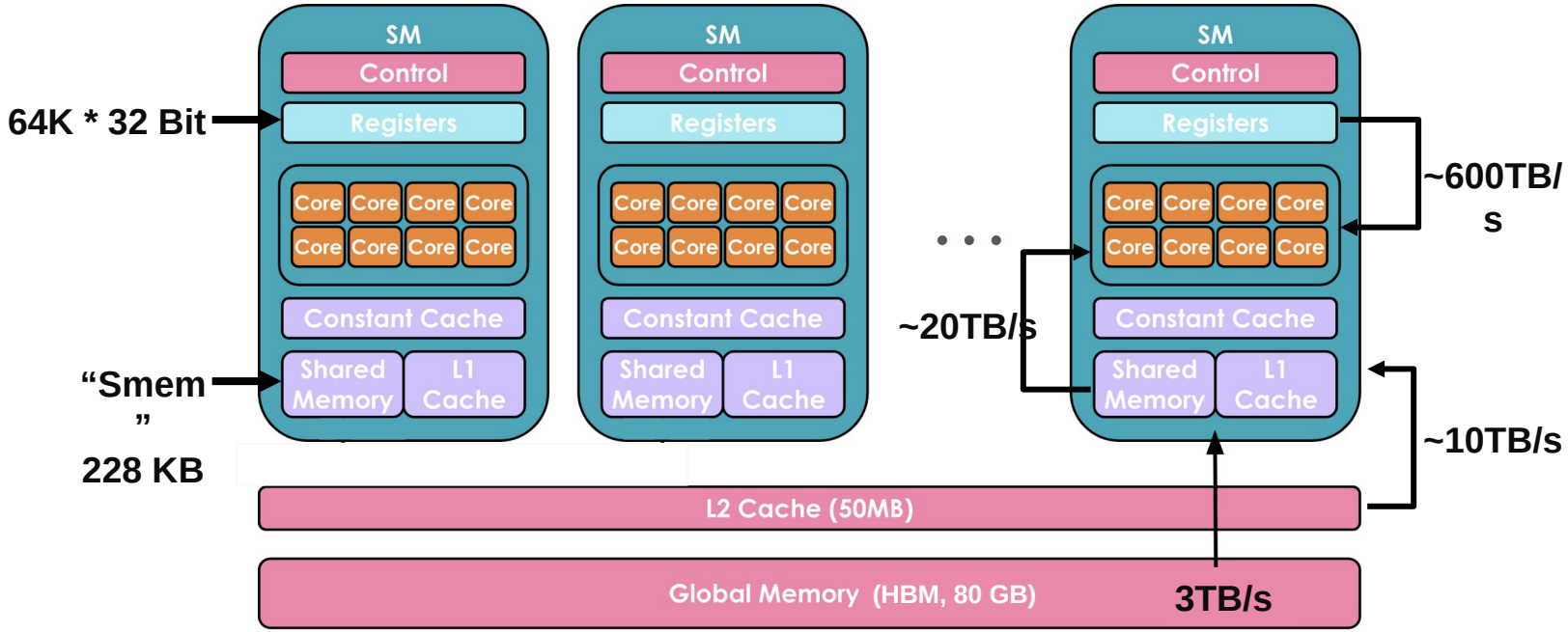
https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/

GPU Hardware Architecture
Zooming In : Streaming Multiprocessors (SMs)

- CUDA Cores
  - Scalar computation
- Tensor Cores
  - Matrix (dense) computation
- Shared / Constant Memory
  - High bandwidth temp buffer

https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/

GPU Memory Hierarchy*

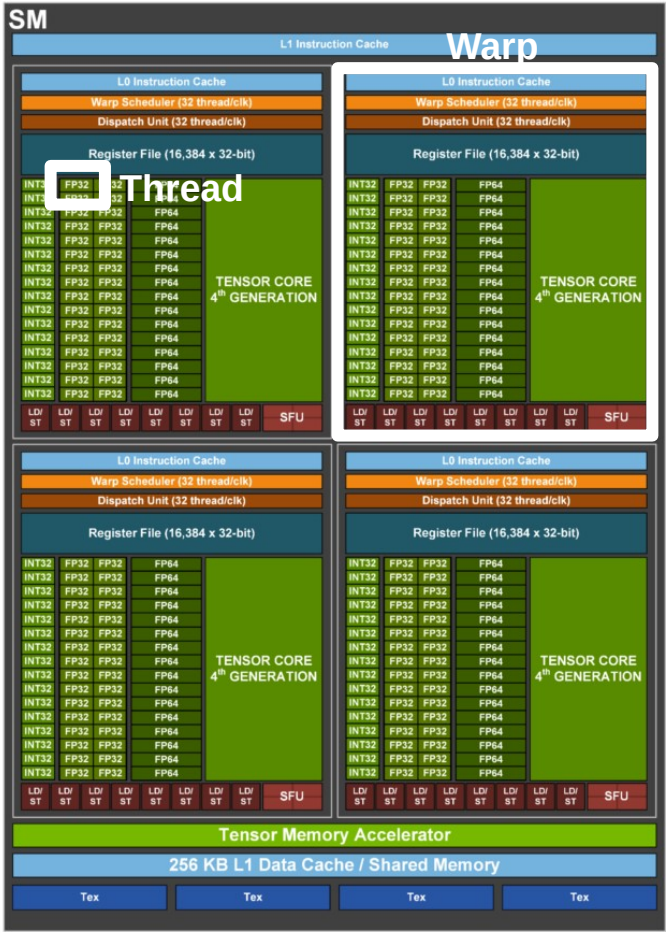


* sizes/latencies for H100

https://nanotron-ultrascale-playbook.static.hf.space/assets/images/diving_primergpu2.svg

GPU Execution Hierarchy - Threads: The True Actor

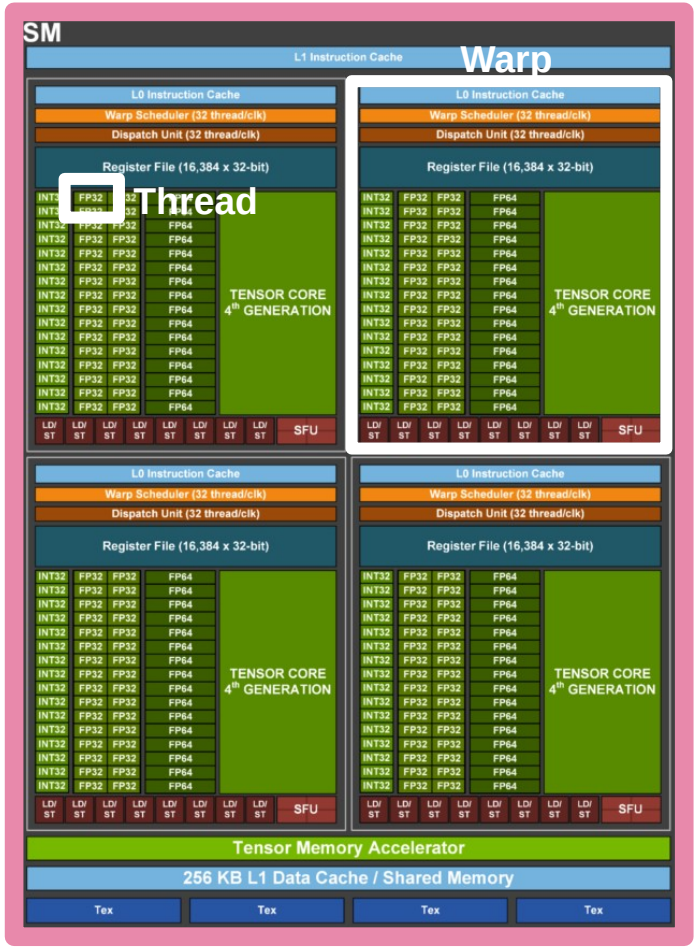
- A thread will use functional units to do the work
- 32 threads form a warp
- Threads in a warp will run in parallel
  - Same instructions
  - Same pace
- Threads exchange data at the register level
  - Shared register files
    - Fast data exchange (600TB/s for H100)



https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/

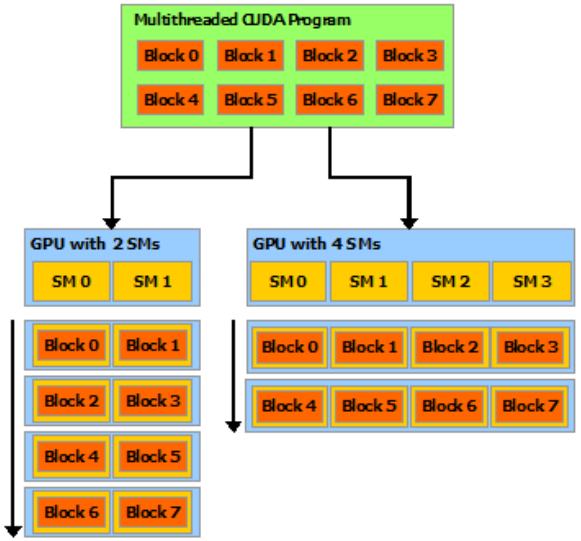
GPU Execution Hierarchy - Thread Blocks

- Thread Blocks are groups of warps
- Blocks are scheduled onto SMs
- 4 warps run on one SM simultaneously
  - You can have up to 32 warps in a block
    - In which case, context switching is needed between warps
- Scheduler swaps warps on and off SM
- Within blocks, warps communicate through the L1 cache/shared memory
  - 20 TB/s on H100

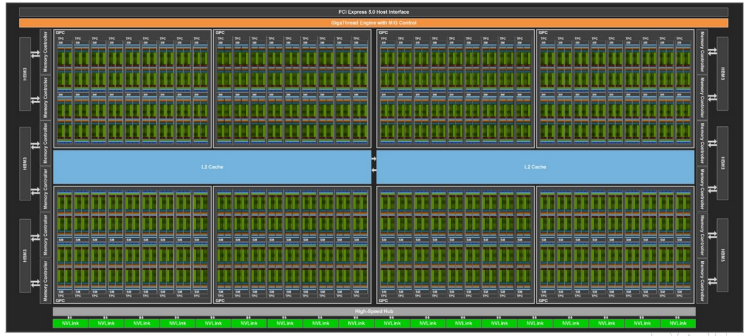


Block

GPU Execution Hierarchy - Kernel



- Function running on a GPU
  - Consists of multiple thread blocks, aka grid
- A kernel can launch more thread blocks than the number of SMs
  - The scheduler will iterate through all blocks
- Block-to-Block communication must be performed via L2 / Global memory
  - 10 TB/s on H100
- Blocks should be designed to operate independently to minimize slow communication
  - E.g., Addition kernel: each block calculates a segment of the output



https://docs.nvidia.com/cuda/cuda-c-programming-guide/

GPU Execution Hierarchy - Streams

- Sequence of operations executed in order
  - By default, CUDA uses a single stream
- Operations from different streams can execute in parallel (provided enough SM resources are available)
  - Kernel execution (e.g., QKV generation) and host-GPU memory transfer (e.g., KV-cache movement)
  - Multiple kernel executions
- Warps from multiple streams can exist concurrently on the same SM
  - Interleaved at warp-level granularity

```
+---------------------+     +---------------------+
| CUDA Stream #0      |     | CUDA Stream #1      |
|-------------------  |     |---------------------|
| Kernel Launch A     |     | Kernel Launch B     |
| └─ Grid             |     | └─ Grid             |
|    └─ Blocks        |     |    └─ Blocks        |
|       └─ Warps      |     |       └─ Warps      |
|          └─ Threads |     |          └─ Threads |
+---------------------+     +---------------------+
            |                           |
            v                           v
   -----------------------------------------
   |          GPU (Streaming Multiprocessors) |
   |       SM0     SM1     SM2     ...        |
   -----------------------------------------
```

GPU Programming Model - Summary

Concept	Definition	Corresponding Architecture	Communication	Limits
Kernel	Function on GPU (Group of Blocks, as defined by a grid)	GPU	L2 / Global memory	(Up to $(2^{32}-1)^3$ Blocks
Thread Blocks	Group of Warps	SM	Shared Memory	Up to 32 warps (1024 threads)
Warp	Group of Threads	SM tiles	Register File	32 threads
Thread	Minimal units that execute instructions	Functional units	Local	Up to 255 registers

Easiest Approach for Addition: Torch

Demo: addition kernel with PyTorch

- Allocating the host tensor
- Move to GPU
- Compute addition
- Move to CPU

Useful tips:

- Use `nvidia-smi` to to check utilization
- If a GPU is busy, you can select an unutilized GPU via
  - `export CUDA_VISIBLE_DEVICES=1`

```python
import torch

def add_tensors(a, b):
    return a + b

if __name__ == "__main__":

    num_elements = 10**9

    # Create two tensors with 1e9 elements each
    tensor1 = torch.rand(num_elements, device='cpu')
    tensor2 = torch.rand(num_elements, device='cpu')

    tensor1 = tensor1.to('cuda')
    tensor2 = tensor2.to('cuda')

    # Add the tensors
    for i in range(10):
        result = add_tensors(tensor1, tensor2)

    result = result.cpu()
    # Print the result
    print("Result of addition:", result)
```

Timing the kernel

- Torch dispatches kernels in a non-blocking way
  - Work dispatched to the GPU and CPU continues execution before GPU finishes
- Using CPU-based timing cannot profile GPU-kernel time
- **We need to use CUDA event!**

Performance profiling

- Profiler can tell the type of kernel and corresponding execution time
- Commonly used profilers
  - Torch.profiler: Good at showing CPU activity, slow processing
  - Nsight Systems (nsys): High performance
    - Install on your computer
  - Nsight Compute (ncu): Tailored for intra-kernel profiling
    - Install on your computer